

AUTOFLOW

AUTOFLOW

Dealer Management System

Angular 17

Supabase

Vercel

Cashfree

TypeScript

Realtime

9

BOOKING STEPS

12

DB TABLES

2

PORTALS

6

SERVICES

Presented by Tej Prakash & Harsimran Kaur

Car rental dealerships managing operations through paper ledgers, spreadsheets, and WhatsApp face a cascade of inefficiencies every day.



No Centralised Booking Record

Advisors take walk-in bookings verbally. Conflicts happen. Double-booked vehicles sit in the parking lot arguing about who called first.



Opaque Revenue Tracking

Advances collected in cash disappear into pockets. No consolidated view of deposits, overdue returns, or extra mileage charges.



Zero Vehicle Visibility

Which car is available right now? Which one is past service threshold? Nobody knows without calling the driver. Fleet management is guesswork.



Manual Quotation Process

Every quote is hand-calculated, typed on a Word doc, printed, and handed over. Errors are common. Leads are lost.

 Angular 17 + TypeScript

Frontend framework with Signals for reactive state, standalone components, and lazy-loaded routes.

Frontend

 PrimeNG + Aura Theme

Enterprise UI component library — tables, dialogs, charts, toast, dropdowns. Zero custom CSS.

UI

 Supabase

PostgreSQL database, Auth (Google OAuth), Realtime subscriptions, Storage, and Edge Functions.

Backend

 Vercel

Hosts the Angular SPA and runs serverless API functions for payment order creation.

Hosting

 Cashfree Payments

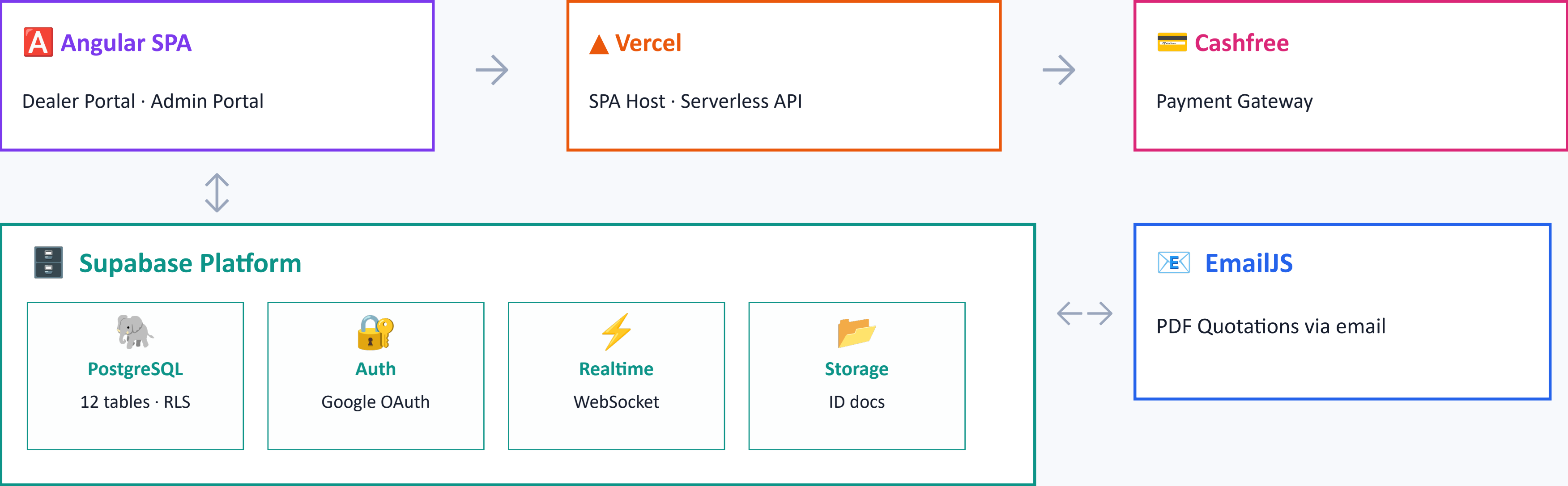
Online payment gateway for UPI, card, and net banking. Integrated via Vercel serverless function.

Payments

 EmailJS + jsPDF

Client-side PDF generation for quotations. EmailJS delivers them to customers — no server needed.

Comms



Key patterns:

- RLS Row-Level Security
- Signals-based state
- SECURITY DEFINER RPCs
- Serverless payments

Role is determined at login from the user_roles table — new Google sign-ins default to Rental Advisor automatically.



Admin

FLEET HEADQUARTERS

- Review and approve / reject booking requests from dealers
- Manage the entire vehicle fleet — add, edit, archive
- View consolidated revenue, analytics, and utilisation charts
- Manage user roles — promote dealers to admin
- Full audit log of every action in the system
- Live booking ledger with Realtime updates



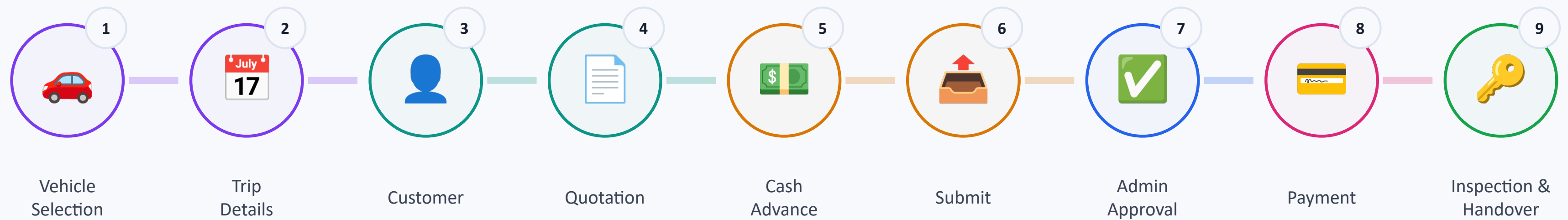
Rental Advisor

WALK-IN BOOKING WIZARD



- 9-step walk-in booking wizard for walk-in customers
- Search returning customers by mobile — auto-fill form
- Generate, preview, and email PDF quotations
- Collect cash or initiate online payment via Cashfree
- Pre-trip inspection form with photo upload
- Personal analytics — revenue, bookings, active vehicles

End-to-End Booking Flow



 Tables written across the full flow:



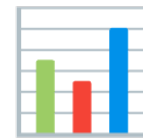
1. Vehicle Selection — Filter by type, availability, and date range
2. Trip Details — Pickup, drop location, and trip purpose
3. Customer — Lookup returning customer by mobile or create new
4. Quotation — Auto-priced PDF with GST, deposit, and advance
5. Cash Advance — Optional pre-approval payment collection
6. Submit — Notify admins of the new booking request
7. Admin Approval — Six DB tables updated atomically
8. Payment — Cashfree gateway or cash collection
9. Inspection & Handover — Photo form; vehicle status set to "rented"



UX

Smart Customer Lookup

Type a mobile number — the system finds the returning customer and auto-fills every field. Debounced search, zero extra clicks.



Core

Auto-Priced Quotations

Base cost, GST, security deposit, and 50% advance are all calculated live from vehicle tier data. Promo code validation included.



Comms

PDF Generation & Email

One-click quotation PDF built client-side with jsPDF, sent via EmailJS. Also sharable via WhatsApp link or SMS.



Realtime

Realtime Notifications

Supabase WebSocket channels push booking updates live. Admins see new requests instantly. Dealers see approvals without refreshing.



Operations

Pre-Trip Inspection

Digital checkout form with photo upload, odometer reading, fuel level, and body damage remarks. Vehicle auto-set to "rented".



Logic

Conflict-Free Availability

Vehicle selection cross-checks all active booking windows against selected dates. A booked car simply doesn't appear in results.



Online — Cashfree Gateway

- 1 Dealer clicks Pay Online — booking must be Admin-approved first.
- 2 Angular calls `createCashfreeOrder()` → Vercel serverless → Cashfree API returns session ID.
- 3 Payment record inserted via `insert_payment_record` RPC (SECURITY DEFINER).
- 4 Cashfree SDK overlay opens. Customer pays via UPI / Card / Net Banking.
- 5 On SDK success → booking activated → navigate to inspection form.



Offline — Cash Collection

Cash can be collected at two points in the flow:

PATH A — Before approval (wizard Step 5)

Dealer collects cash → payment row inserted → booking submitted → Admin approves → system detects existing paid row → auto-activates booking.

PATH B — After approval (payment page)

Booking approved → dealer collects cash → `recordOfflineAdvancePayment()` → `markBookingInService()` → navigate to inspection.

Security note: server-side Cashfree verification endpoint is planned for v2.

Database Design — 12 Tables

bookings Central record. Every other table relates back to this one. 9 readers 7 writers	customers Mobile as unique key. Upsert on duplicate — never duplicates. 5 readers 3 writers	vehicle Fleet records. Status: available → booked → rented → dirty. 5 readers 6 writers	quotations 1:1 with bookings via unique constraint. Holds all pricing. 3 readers 8 writers
payments Online (Cashfree via RPC) or offline cash. Status: initiated → paid. 3 readers 3 writers	notifications In-app alerts. Realtime subscribed — appear without refresh. 1 reader 3 writers	audit_logs Immutable trail. Every important action is stamped here. 1 reader 1 writer	user_roles Admin or dealer. Auto-created on first Google login. 3 readers 2 writers
vehicle_tiers Pricing config — daily rate, deposit, mileage limits per tier. 4 readers admin only	dealers Company name, GST, phone, address. Upserted on signup. 2 readers 2 writers	inspections Vehicle checkout form. Written once per booking at handover. UI soon 1 writer	bulk_bookings Group fleet booking headers. Schema ready, UI in progress. 1 reader wip



Admin View

6 pages

₹2.4L

Monthly Revenue

87%

Fleet Utilisation

34

Active Bookings

12

Pending Approvals

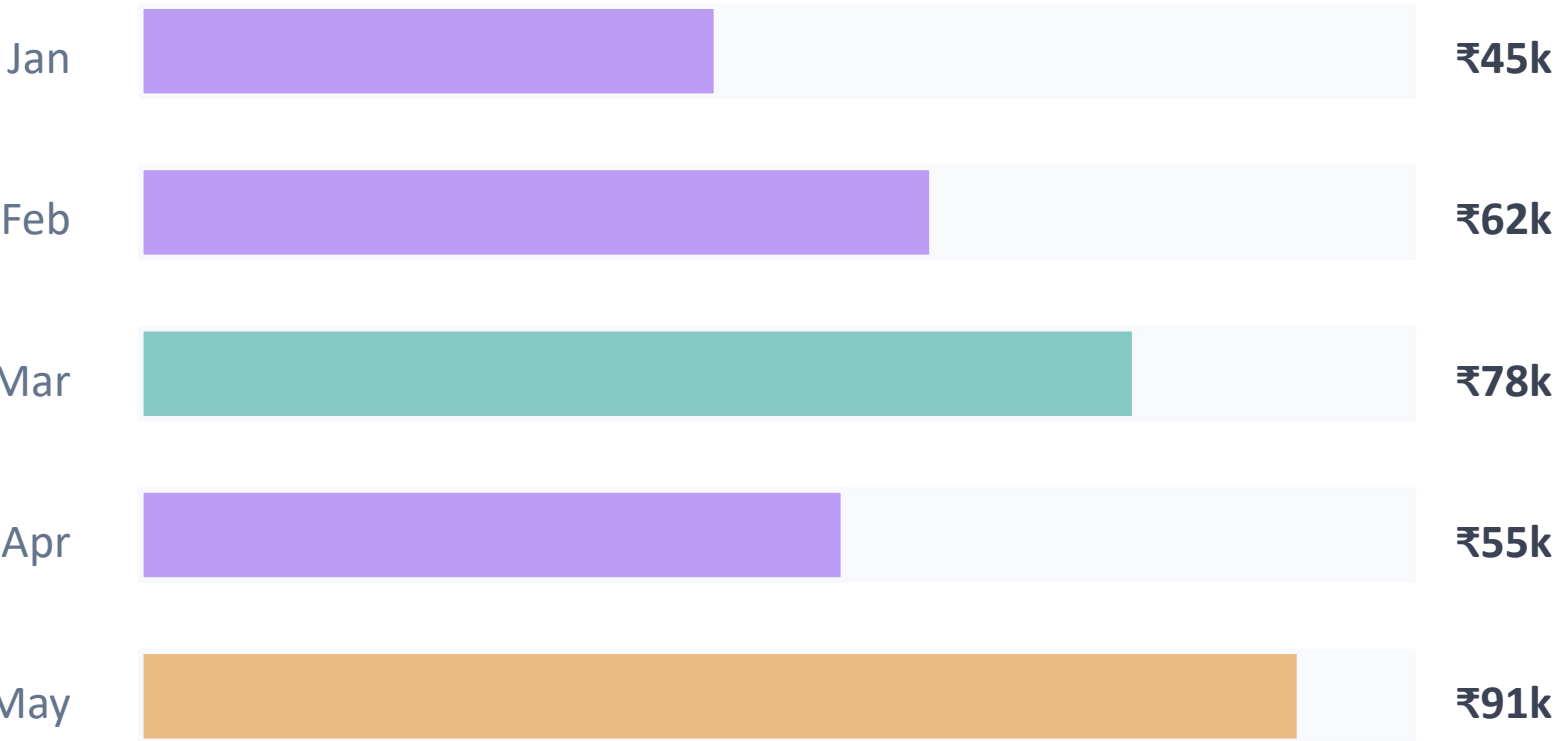
Annual revenue pie · Monthly bar charts · Vehicle utilisation · Booking-to-revenue breakdown · Advisor performance · Full audit log with CSV export



Dealer View

Personal only

Revenue by month (this dealer only):



Also tracks: frequency of bookings · active vehicles · drop-off bookings · users onboarded · exportable as CSV

Supabase Realtime keeps every screen in sync via persistent WebSocket connections — no polling, no page refreshes.

● Notifications channel

Subscribed per user. New notification rows fire the callback instantly — the bell icon updates live.



New booking request — Toyota Fortuner, 3 nights, ₹12,000

just now · booking_id: BK-20250601-0087



Booking approved — Proceed to payment collection

2 min ago · booking_id: BK-20250601-0083

Code: `supabase.channel().on('postgres_changes',...).subscribe()`

● Booking changes channel

Admin booking ledger subscribes to ALL booking changes. Status updates appear without refreshing the table.



Status changed: pending → approved

just now · Dealer: Rahul Mehta



Vehicle handed over — Honda City → status: rented

5 min ago · odometer locked at 24,810 km

Code: `subscribeToMyBookingChanges(userId)` for personal feed

Complex Multi-Table Writes

Problem

A single walk-in booking creation needs to atomically write to customers, bookings, quotations, and vehicle — partial failures would leave orphan records.

Solution

`createWalkInQuotation()` sequences all writes with early-exit error handling. Each step checks the previous before proceeding, keeping state consistent.

Duplicate Customer Prevention

Problem

Walk-in customers often don't remember if they've booked before. Standard inserts would create duplicate rows — different IDs, same person.

Solution

`upsertCustomerByMobile()` catches Postgres error code 23505 (duplicate key on mobile) and fetches the existing row instead of creating a new one.

RLS vs. Privileged Writes

Problem

Row-Level Security prevents dealers from writing payment records directly — but they need to insert their own payment rows. Bypassing RLS for all users is insecure.

Solution

SECURITY DEFINER PostgreSQL functions (`insert_payment_record`) run with elevated privileges only for specific, controlled operations.

Shared State Across Routes

Problem

The routed booking flow (customer-details → quotation → payment) spans three Angular components that all need the same booking data without re-fetching.

Solution

`BookingFlowService` uses Angular Signals to hold shared state. `loadBooking()` caches the result — subsequent route steps read from memory, not the DB.

Security — Immediate**Payment Verification**

- Wire api/confirm-cashfree-payment.js to verify orders server-side after SDK resolves
- Move all API keys to environment variables
- JWT claims for role — remove DB round-trip on every auth check

v2 — Near-term**Vehicle Return Flow**

- Return inspection form with check-in photos
- Auto-calculate extra mileage, fuel discrepancy, late penalty
- Security deposit release with final invoice generation
- Maintenance threshold → route to service, not available

v3 — Expansion**Platform Growth**

- Customer-facing booking portal (self-service)
- Bulk / fleet booking UI
- Digital rental agreement with e-signature
- WhatsApp Business API integration
- Mobile app (Capacitor / Ionic wrapper)

 Live on Vercel

See It Live

The full system is deployed and functional

 delar-management.vercel.app

9

FLOW STEPS

12

DB TABLES

6

SERVICES

2000+

LINES OF LOGIC

∞

REALTIME

Thank you — Questions welcome

• Tej Prakash • Harsimran Kaur